MedSoft HIS — учебная госпитальная информационная система

Лабораторные 1 и 2: сервер API, веб-клиент (раздельные рабочие места администратора и медработника) и база данных, плюс базовая безопасность: TLS 1.3 с собственным CA, Argon2id, JWT (EdDSA) с access/refresh, TOTP для администратора, роли и аудит.

Запуск

Сертификаты и закрытые ключи в репозиторий не попадают: при первом запуске `start.bat` / `start.sh` выпускают их сами (`tools/make_certs.py`). После этого нужно один раз добавить новый `certs/ca.crt` в доверенные.

Windows

```
# 1. Окружение (один раз)
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt

# 2. Запуск: выпуск сертификатов при первом старте, два окна (API и клиент), браузер
start.bat

# 3. Доверие к учебному корневому CA (один раз, Windows спросит подтверждение)
certutil -user -addstore Root certs\ca.crt
```

macOS

Нужен Python 3.10 или новее (`brew install python@3.12`).

```
chmod +x start.sh   # если git не сохранил право на запуск
./start.sh          # окружение, зависимости, сертификаты, запуск; остановка Ctrl+C

# доверие к учебному CA (один раз, macOS спросит пароль)
security add-trusted-cert -r trustRoot -k ~/Library/Keychains/login.keychain-db certs/ca.crt
```

То же вручную: открыть `certs/ca.crt` в «Связке ключей» → «Доверие» → «Всегда доверять». После добавления CA перезапустите браузер.

- Клиент: https://localhost:8444/ → «Администратор» (`/admin/`) или «Медработник» (`/staff/`)
- API: https://localhost:8443/ (Swagger: https://localhost:8443/docs)

Chrome, Edge и Safari берут доверенные сертификаты из хранилища системы. Firefox использует своё хранилище: Настройки → Приватность и защита → Сертификаты → Импортировать `certs/ca.crt`.

Удалить учебный CA из доверенных: Windows — `certutil -user -delstore Root "MedSoft 367627 Root CA"`, macOS — удалить сертификат в «Связке ключей».

Первый вход

1. `/admin/` → мастер настройки: пароль администратора (от 12 символов, заглавные, строчные, цифры) → QR-код для Google Authenticator / Яндекс Ключа → ввод кода. После этого вход только по паролю и коду.
2. «Справочник диагнозов» → «Загрузить реестр».
3. «Медработники» → «Добавить» → роль `doctor` → в карточке «Создать аккаунт». Пароль показывается один раз.

4. `/staff/` → вход по выданным логину и паролю.

Проверки

```
.venv\Scripts\python.exe tools\show_cert.py          # какие сертификаты отдаёт сервер (Subject/Issuer/SAN)
.venv\Scripts\python.exe tools\smoke_test.py         # 65 сквозных проверок API (нужен сервер на ПУСТОЙ базе)
```

Для smoke-теста запустите сервер на отдельной базе, чтобы не трогать рабочую:

```
$env:HIS_DB_URL = "sqlite:///C:/Users/churk/med_soft/data/test.db"; .venv\Scripts\python.exe -m server
```

Архитектура

```
Браузер ──TLS 1.3──> client/serve.py  :8444   статика (HTML/CSS/JS), заголовок CSP
   │
   └──────TLS 1.3──> server (FastAPI) :8443   REST + WebSocket ──> SQLite (SQLAlchemy)
```

Сервер и клиент — два независимых процесса на разных origin; API разрешает клиенту доступ через CORS. Клиент без фреймворков (ES-модули), сборка не нужна.

Путь	Что там
`server/rules.py`	**все ограничения полей** (длины, enum ролей и пола)
`server/models.py`	схема БД
`server/schemas.py`	схемы запросов/ответов (валидация pydantic)
`server/security/permissions.py`	**таблица «роль → права»**
`server/security/tokens.py`	JWT EdDSA, ротация и отзыв refresh
`server/security/passwords.py`	Argon2id, генератор паролей
`server/security/totp.py`	TOTP RFC 6238, QR-код
`server/audit.py`	запись аудита
`server/emr_service.py`	ЭМК: состояние на момент времени, пакет изменений
`server/routers/*`	обработчики API
`client/web/admin`, `client/web/staff`	два раздельных интерфейса
`tools/make_certs.py`	выпуск CA и серверного сертификата

Если на защите попросят поменять поле

- **Длина или ограничение существующего поля** → `server/rules.py` (одно место).
- **Новое поле** → `models.py` (колонка) + `schemas.py` (вход/выход) + форма в клиенте. Миграций нет: после изменения схемы удалите `data/his.db`, и таблицы создадутся заново.
- **Кому что разрешено** → `ROLE_PERMISSIONS` в `security/permissions.py`.

Решения, которые стоит уметь объяснить

Сертификаты: Issuer и Subject. Issuer — кто подписал, Subject — кому выдан. У корневого CA они совпадают (самоподписанный): `CN=MedSoft 367627 Root CA, OU=MedSoft 2026/27 367627, O=ITMO`. У серверного сертификата Issuer — это Subject CA, а Subject — `CN=localhost` с тем же O/OU. Браузер проверяет имя хоста по SubjectAltName (`localhost`, `127.0.0.1`, `::1`). CA: `BasicConstraints CA:TRUE, pathlen:0`, KeyUsage `keyCertSign, cRLSign`. Сервер: `CA:FALSE`, EKU `serverAuth`. Ключи ECDSA (P-384 у CA, P-256 у сервера). Оба сервера принимают только TLS 1.3.

Пароли. Argon2id (RFC 9106): память-ёмкий KDF, соль и параметры хранятся внутри строки хэша. Если логин не найден, всё равно проверяется хэш-пустышка: время ответа не выдаёт, существует ли логин. Ответ на любой неуспешный вход одинаковый, а точная причина пишется только в аудит.

JWT. Подпись EdDSA (Ed25519): закрытым ключом подписывает только сервер API, открытым может проверять кто угодно. Access-токен живёт 5 мин и несёт роль; права проверяются при каждом запросе по токену, без обращения к БД. Refresh живёт 8 ч и учитывается в таблице `refresh_tokens`. При обновлении выдаётся новая пара, а старый refresh отзывается (ротация). Если отозванный при ротации refresh предъявят повторно, это признак кражи: отзываются все токены субъекта, событие пишется в аудит. При деактивации аккаунта, смене пароля или роли refresh-токены отзываются, их количество указывается в аудите. Компромисс: уже выданный access-токен действует до своего истечения (до 5 минут).

TOTP. RFC 6238: HMAC-SHA1, 6 цифр, шаг 30 с, допуск ±1 шаг. Сервер запоминает номер последнего использованного шага, поэтому один и тот же код нельзя применить дважды. Настройка двухшаговая: секрет хранится как «ожидающий» и становится рабочим только после верного кода. Шаги связаны одноразовым `setup_id`.

Аудит. `actor` — это `<тип>:<id>`: `user:<uuid аккаунта>`, `admin:<uuid>`, `anonymous:<ip>` (неаутентифицированный, например неудачный вход), `system:server`. UUID не переиспользуется, поэтому субъект однозначен даже после переименований; `actor_name` — снимок для чтения человеком. Запись аудита идёт в той же транзакции, что и изменение данных. Таблица только дописывается: в API нет методов изменения, а SQLite-триггеры запрещают UPDATE/DELETE. В реальном времени события доставляются по WebSocket (опрос БД раз в 1 с, задержка ≤ 2 с). Токен передаётся первым сообщением, а не в URL, проверяются право `admin` и Origin. Когда access истекает, клиент присылает новый токен, иначе соединение закрывается.

История ЭМК. Каждый диагноз или назначение хранит интервал жизни `[added_at, removed_at)`. Состояние на момент T — это записи, где `added_at ≤ T < removed_at`. Каждое действие дополнительно пишется в `emr_history` (кто, когда, что, полные данные). Редактирование назначения — это отмена старого и новое с `replaces_id`. Изменения из интерфейса уходят одним пакетом и применяются атомарно. Оптимистическая блокировка (`version`) не даёт двум врачам затереть изменения друг друга: второй получит 409.

Удаление пациента. Запрещено при наличии ЭМК на двух уровнях: проверка в API и внешний ключ `ON DELETE RESTRICT` в БД.

Клиент. Access-токен хранится только в памяти страницы, refresh — в `sessionStorage` вкладки. Компромисс: так перезагрузка не выкидывает из системы, но токен доступен JS этого origin. Защита от XSS: весь текст вставляется через `textContent`, а строгий CSP запрещает inline-скрипты и чужие источники. Одновременно выполняется только один refresh, чтобы не сработала защита от повторного использования.

Роли.

Право	doctor	nurse	laboratory-technician	other	admin
Смотреть пациентов и ЭМК	✓	✓	✓	✓	
Создавать/менять/удалять пациентов, создавать ЭМК	✓	✓			
Разблокировать и редактировать ЭМК	✓				
Интерфейс администратора					✓

Служебные поля

`created_at/updated_at` — когда запись появилась и менялась; `staff.is_deleted/deleted_at` — логическое удаление; `emrs.version` — оптимистическая блокировка; снимки `code/name` диагноза в ЭМК — справочник может измениться, а история должна показывать то, что было; `audit_events.seq` — порядок и инкрементальная доставка по WebSocket; `admin_account.singleton` — гарантия одного администратора на уровне БД; `admin_account.totp_last_step` — защита от повторного использования TOTP-кода.